DEV SANSKRITI VISHWAVIDYALAYA (DSVV), HARIDWAR

Department of Computer Science · BCA VII Semester

Node.js Practical Lab Manual & Viva Handbook

Complete Syllabus: Lab 01 to Lab 08 · Architecture, Source Code Breakdown & Teacher Viva Q&A

Candidate Name:	Bhaskar Mall	**Scholar / Roll No:**	23145005
Course & Sem:	BCA VII Semester	**Subject Code:**	CS403NOD — Node.js
Institution:	Dev Sanskriti Vishwavidyalaya (DSVV), Haridwar		

This exhaustive document prepares the student for both live terminal code evaluation, architecture defense, line-by-line code explanation, error handling troubleshooting, and rapid-fire oral viva assessments conducted by faculty examiners.

- ✓ All 8 Labs Covered
- ✓ Core Architecture (libuv, V8, Event Loop)
- ✓ Async Paradigms (Callbacks, Promises, Async/Await)
- ✓ fs, http, events, child_process Modules
- ✓ Top 30 Faculty Viva Questions & Answers
- ✓ Command-Line & Debugging Cheatsheet

1. Core Node.js Architecture & Execution Model

Faculty examiners frequently begin viva evaluations by questioning the internal architecture of Node.js to verify if the student understands how JavaScript runs outside the browser and handles concurrent requests.

Q1. What exactly is Node.js, and what are its two core internal dependencies?

Answer: Node.js is **not a language or framework**; it is an open-source, cross-platform **JavaScript runtime environment** built on Google Chrome's **V8 JavaScript Engine** and written in C++.
The two foundational C/C++ dependencies are:

- **Google V8 Engine:** Compiles JavaScript source code directly into native machine code (Just-In-Time compilation) instead of interpreting bytecode.
- **libuv (C library):** A high-performance, cross-platform C library that provides the **Event Loop**, non-blocking asynchronous I/O polling, and a background **Thread Pool** (default 4 threads) for blocking system tasks like file I/O, DNS lookup, and cryptographic operations.

Q2. How does Node.js handle thousands of concurrent requests if JavaScript is single-threaded?

Answer: Node.js executes JavaScript on a **single main thread**, but it achieves high concurrency using an **Event-Driven, Non-Blocking I/O model** supported by the underlying OS kernel (via `epoll` on Linux, `kqueue` on macOS, `IOCP` on Windows) and `libuv` thread pool.
When an I/O request (e.g. database query, reading a file, network request) arrives, Node.js offloads the operation to the operating system or libuv thread pool and immediately registers a callback. The main thread is freed up instantly to handle new incoming HTTP requests. When the I/O completes, its callback is placed in the Event Loop queue and processed when the main stack is empty.

Q3. What are the six phases of the Node.js Event Loop in order of execution?

Answer:

1. **Timers Phase:** Executes callbacks scheduled by `setTimeout()` and `setInterval()` whose threshold elapsed.
2. **Pending / I/O Callbacks:** Executes deferred system callbacks (e.g., TCP error reports).
3. **Idle, Prepare:** Internal libuv bookkeeping.
4. **Poll Phase:** Retrieves new I/O events, executes I/O related callbacks (almost all code except timers, close, check); blocks and waits for connections if queue is empty.
5. **Check Phase:** Executes callbacks invoked by `setImmediate()`.
6. **Close Callbacks:** Handles cleanup, e.g., `socket.on('close', ...)`.

Special Microtask Queues: `process.nextTick()` runs immediately after the current operation finishes, before the event loop advances to the next phase, followed by Promise microtasks.

Q4. Browser JavaScript vs Node.js — What are the key differences?

Feature	Browser Environment	Node.js Environment
Root Global Object	`window`, `document`	`global`, `process`
DOM & UI	Direct access to DOM elements, alerts	No DOM, no UI; runs headlessly on the server
File System & OS	Blocked by sandboxing for security	Direct full access via `fs`, `os`, `path`
Module System	ES Modules (`import/export`)	CommonJS (`require/module.exports`) & ESM

2. Comprehensive Lab-by-Lab Viva & Practical Breakdown

LAB 01 — Node.js Setup, Variables, Data Types & Base HTTP Server PORT 3000 / CLI

Primary Files: `Lab-01/app.js`, `Lab-01/server.js`

Objective: Prove working Node.js runtime, inspect variable data types with `typeof`, understand scope (`var`, `let`, `const`), and launch the most basic HTTP listener.

Teacher Question: Explain the code inside Lab-01's server.js and why res.end() is mandatory.

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js Lab 01!');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Explanation to give:

1. `require('http')` loads Node's built-in HTTP module without requiring npm install.
2. `http.createServer()` returns an HTTP server instance and takes a callback function receiving two stream objects: `req` (IncomingMessage) and `res` (ServerResponse).
3. `res.writeHead(200, { 'Content-Type': 'text/plain' })` sets the HTTP status code (200 OK) and HTTP response headers before any body bytes are transmitted.
4. **Why `res.end()` is mandatory:** `res` is a writable stream. Calling `res.end()` signals to the client browser and operating system that all response headers and body data have been sent. If omitted, the client request remains hanging indefinitely until it times out.

LAB 02 — HTTP Web Server & Custom Routing ROUTING / JSON / STATUS CODES

Primary Files: `Lab-02/server.js`

Objective: Build a multi-endpoint HTTP server handling multiple routes (`/`, `/about`, `/college`, `/profile`, `/students`, `* 404`) and return JSON.

Teacher Question: How do you implement routing without Express in native Node.js?

Answer: Native Node.js inspects the `req.url` and `req.method` properties on the incoming request object using a `switch-case` or `if-else` statement:

```
const server = http.createServer((req, res) => {
  const path = req.url.toLowerCase();

  if (path === '/' || path === '/home') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Welcome to DSVV Lab 02</h1>');
  } else if (path === '/students') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify([{ id: 1, name: 'Bhaskar', course: 'BCA' }]));
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Route Not Found', statusCode: 404 }));
  }
});
```

Crucial Viva Point: Always handle the default / fallback case with HTTP Status **404 (Not Found)**, or the server will hang on unhandled paths!

LAB 03 & 04 — Dynamic REST API, URL Query Params, Sorting & Node Modules

REST API / URL / QUERY PARSER

Primary Files: `Lab-03/student-server.js`, `Lab-04/advanced-server.js`, `Lab-04/app.js`

Objective: Filter, validate, search, and sort structured in-memory datasets using URL query strings (e.g. `?course=BCA&minMarks=70&sort=name&order=asc`).

Teacher Question: How did you extract and sanitize URL query parameters in Lab 04?

Answer: Using Node's native `url` module with the parse method and passing `true` as the second parameter:

```
const url = require('url');
const parsedUrl = url.parse(req.url, true); // true parses query string into object
const query = parsedUrl.query; // e.g. { course: 'BCA', minMarks: '70' }
const pathname = parsedUrl.pathname;

if (pathname === '/students') {
  let results = [...studentDatabase];

  // 1. Filter by course (case-insensitive)
  if (query.course) {
    results = results.filter(s => s.course.toLowerCase() === query.course.toLowerCase());
  }

  // 2. Numeric validation for minMarks
  if (query.minMarks) {
    const min = parseFloat(query.minMarks);
    if (!isNaN(min)) {
      results = results.filter(s => s.marks >= min);
    }
  }

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ total: results.length, data: results }));
}
```

Teacher Question: What is the difference between Core Modules, Local Modules, and Third-Party Modules?

Answer:

- **Core Modules:** Built right into the Node.js binary (e.g., `http`, `fs`, `path`, `events`, `os`). Loaded directly by name: `require('fs')`. No installation needed.
- **Local Modules:** Files created by the developer in the project (e.g., `./modules/logger.js`). Loaded with relative file paths: `require('./logger')`.
- **Third-Party Modules:** Installed from the npm registry (e.g., `slugify`). Stored in `node_modules/`. Loaded by package name: `require('slugify')`.

LAB 05 — Asynchronous Paradigms: Callbacks, Promises & Async/Await

ASYNC / PROMISES / PROMISE.ALL

Primary Files: `Lab-05/callback-version.js`, `Lab-05/promise-version.js`, `Lab-05/chaining-version.js`, `Lab-05/async-await-version.js`, `Lab-05/concurrent-orders.js`

Objective: Understand and compare the progression of asynchronous JavaScript using a real-world multi-step Food Delivery Tracking simulation (Place Order → Prepare Food → Dispatch → Deliver).

Teacher Question: Compare Callbacks, Promises, and Async/Await. What is Callback Hell?

Answer:

- **Callbacks:** Functions passed as arguments to other functions to be called upon completion. Leads to *Callback Hell* (Pyramid of Doom) when nesting sequential tasks, making error handling and code readability extremely poor.
- **Promises:** ES6 objects representing the eventual completion (fulfillment) or failure (rejection) of an asynchronous operation. Has 3 states: *Pending*, *Fulfilled*, *Rejected*. Solves nesting via `.then()` chaining and centralized `.catch()`.
- **Async/Await:** Syntactic sugar built on top of Promises (ES2017). Allows writing asynchronous code that looks and behaves like synchronous code. Uses standard `try...catch` blocks for error handling.

```
// Modern Async/Await pattern from Lab 05:
async function processDeliveryOrder(orderId) {
  try {
    const order = await placeOrder(orderId);
    const cooked = await prepareFood(order);
    const rider = await dispatchRider(cooked);
    const delivered = await deliverFood(rider);
    console.log('Delivered successfully:', delivered);
  } catch (err) {
    console.error('Delivery step failed:', err.message);
  }
}
```

Teacher Question: What is Promise.all() and when is it preferred over sequential awaits?

Answer: `Promise.all([p1, p2, p3])` takes an iterable of promises and executes them **concurrently (in parallel)**. It resolves when *all* promises resolve, or rejects immediately when *any* promise rejects (fail-fast behavior).
When preferred: When asynchronous tasks are **independent** of one another (e.g. preparing 3 separate food items or reading 3 independent files), sequential `await` takes `t1 + t2 + t3` seconds, whereas `Promise.all()` runs them in parallel taking only `max(t1, t2, t3)` seconds.

LAB 06 — File System (fs) Module & CLI Notes Application

FS / FS.PROMISES / PROCESS.ARGV

Primary Files: `Lab-06/fs-demo.js`, `Lab-06/read-async.js`, `Lab-06/read-sync.js`, `Lab-06/add-note.js`, `Lab-06/read-notes.js`

Objective: File manipulation in Node.js: synchronous vs asynchronous methods, `fs.promises` API, file deletion (`unlink`), and command-line arguments (`process.argv`).

Teacher Question: Why does fs.readFile() not block your program while fs.readFileSync() does?

Answer (Direct from Lab 06 Reflection):
`fs.readFile()` is asynchronous. Node.js initiates the OS read request and offloads it to the libuv thread pool, immediately continuing execution of the next lines in the JavaScript script without waiting. The callback is invoked only when the file reading completes and the event loop reaches the poll phase.
In contrast, `fs.readFileSync()` is synchronous; it halts and blocks the single JavaScript thread completely until the entire file is fetched from storage into memory. No other requests or lines can execute during this freeze.

Teacher Question: In the Notes App, what happens if two users execute add-note.js simultaneously?

Answer (Direct from Lab 06 Reflection Question 2):
A **Race Condition** occurs. When two processes execute concurrent write/append operations on the exact same physical text file without file-locking mechanisms, their write operations may interleave, leading to corrupted records, out-of-order writes, or partial write errors. In high-concurrency production systems, an atomic database (e.g. PostgreSQL, MongoDB, Redis) or transactional queue must be used instead of raw text file appending.

Teacher Question: How do you read command line arguments in Node.js?

Answer: Via the global `process.argv` array:
`process.argv[0]` = Absolute path to Node binary (`node.exe`).
`process.argv[1]` = Absolute path to executing script file.
`process.argv[2]` and onwards = User-provided CLI arguments.
Example: `node add-note.js "Complete Node.js Lab"` → `process.argv.slice(2).join(' ')` gets the full string.

LAB 07 — EventEmitter Architecture & Custom Event Systems

EVENTS / PUBSUB / LISTENER LIFECYCLE

Primary Files: `Lab-07/emitter.js`, `Lab-07/07-practical-example/app.js`

Objective: Master Node.js Publisher-Subscriber architecture using the built-in `EventEmitter` class from the `events` module.

Teacher Question: Explain EventEmitter methods: on, emit, once, removeListener.

Answer:

- `emitter.on(eventName, listener)` : Registers an event listener callback function that triggers *every single time* the specified event is emitted.
- `emitter.emit(eventName, ...args)` : Synchronously calls each of the listeners registered for the event named `eventName` , in the order they were registered, passing the supplied arguments to each.
- `emitter.once(eventName, listener)` : Adds a one-time listener function. The next time the event is triggered, this listener is removed and then invoked.
- `emitter.removeListener(eventName, listener)` : Unregisters a specific listener function from the listener array for the specified event to prevent memory leaks.

```
const EventEmitter = require('events');

class OrderTracker extends EventEmitter {}
const tracker = new OrderTracker();

// Multi-listener demo
tracker.on('orderPlaced', (data) => console.log('SMS sent to:', data.customer));
tracker.on('orderPlaced', (data) => console.log('Kitchen preparing:', data.item));
tracker.once('orderDelivered', (data) => console.log('Single-fire delivery record'));

tracker.emit('orderPlaced', { orderId: 101, customer: 'Bhaskar', item: 'Laptop' });
```

Critical Warning for Viva: If an EventEmitter emits an `'error'` event and there are no listeners subscribed to `'error'` , Node.js throws an unhandled exception and crashes the entire process! Always register `emitter.on('error', handler)` .

LAB 08 — Integrated Monolithic Lab Server (Local + Live Cloud Deployment)

INTEGRATION / CHILD_PROCESS / LOGGER

Primary Files: `Lab-08/server.js` , `Lab-08/labs.js` , `Lab-08/modules/logger.js`

Objective: Unify all previous 7 labs into a single monolithic production server without modifying or rewriting original code. Reuses HTTP server handlers via `module.exports` delegation, executes script labs using sandboxed `child_process.execFile` , records asynchronous file logs via `EventEmitter` , supports cloud deployment (Render), and serves live health & dashboard metrics.

Teacher Question: How did you integrate all previous labs without rewriting them?

Answer:

1. **Server Labs (Lab 01, Lab 02):** The integrated server dynamically imports the exported handler function via `require(labPath)` . When a request hits `/labs/:id/app/...` , the integrated server strips the prefix from `req.url` and delegates execution to the old lab's request handler: `handler(req, res)` .
2. **Script Labs (Lab 03 to Lab 07):** The integrated server executes the target script using Node's native `child_process.execFile()` wrapped in `util.promisify` with a 5000ms timeout guard:

```
const { execFile } = require('child_process');
const { promisify } = require('util');
const execFileAsync = promisify(execFile);

const result = await execFileAsync(process.execPath, [labFilePath], { timeout: 5000 });
res.end(JSON.stringify({ ok: true, output: result.stdout }));
```

Teacher Question: How does the custom EventEmitter logging system in Lab 08 work?

Answer: Inside `modules/logger.js` , a custom class extending `EventEmitter` is instantiated. For every incoming HTTP request, the main server emits: `logger.emit("request", req.method, req.url);`
The logger listener formats an ISO timestamped entry and asynchronously writes it to disk:

```
logger.on("request", (method, requestUrl) => {
  const line = new Date().toISOString() + " " + method + " " + requestUrl;
  console.log(line);
  fs.appendFile(LOG_FILE, line + "\n", (err) => {
    if (err) logger.emit("error", err);
  });
});
```

This decouples the request-handling logic from disk file I/O operations.

Teacher Question: What is child_process.execFile vs exec vs spawn vs fork?

Answer:

- `execFile` : Directly invokes an executable file without spawning an intermediate system shell. Safer against command injection attacks and faster. (Used in Lab 08).
- `exec` : Spawns an entire system shell (e.g. bash/cmd) and executes the command string. Vulnerable to shell injection if user inputs are concatenated.
- `spawn` : Spawns a child process and streams large amounts of data via `stdout` / `stderr` streams without buffering everything in memory.
- `fork` : A specialized instance of `spawn` designed specifically to create new Node.js V8 processes with built-in Inter-Process Communication (IPC) channels.

3. Top 25 Rapid-Fire Faculty Viva Questions & Answers

1. What is the difference between require() and import?

`require()` is synchronous and part of the CommonJS module system used traditionally by Node.js. `import` is asynchronous, static, and part of the ECMAScript Module (ESM) standard.

2. What is module.exports vs exports?

`module.exports` is the actual object returned when someone calls `require()`. `exports` is merely a shorthand variable reference pointing to `module.exports`. If you reassign `exports = something`, you break the reference; always assign to `module.exports`.

3. What is the purpose of package.json and package-lock.json?

`package.json` contains project metadata, scripts, and semantic version ranges for dependencies. `package-lock.json` records the *exact version tree and cryptographic hash* of every installed nested dependency to guarantee deterministic builds across machines.

4. What does the caret (^) and tilde (~) mean in dependency versioning?

`^1.6.6` allows minor and patch updates (e.g. 1.6.7 or 1.7.0, but not 2.0.0). `~1.6.6` allows only patch updates (e.g. 1.6.7, but not 1.7.0).

5. What is the Event Loop Poll phase and why is it important?

The poll phase calculates how long it should block and poll for incoming I/O, then processes the events in the poll queue. If the queue is empty, it checks for `setImmediate()` or waits for timers to expire.

6. What is the difference between process.nextTick() and setImmediate()?

`process.nextTick()` runs immediately after the current operation finishes on the call stack, before the event loop advances. `setImmediate()` runs during the *Check phase* of the event loop.

7. What is a Stream in Node.js and what are the 4 stream types?

A stream is an abstract interface for handling reading or writing continuous chunks of data without loading the entire payload into RAM. The 4 types are: **Readable** (e.g. `fs.createReadStream`), **Writable** (e.g. `res`, `fs.createWriteStream`), **Duplex** (e.g. TCP net.Socket), and **Transform** (e.g. `zlib.createGzip`).

8. What is the difference between res.write() and res.end()?

`res.write()` sends chunks of response body data over the socket without closing the connection. `res.end()` sends the optional final chunk and signals to the client that the HTTP response is complete.

9. What is an error-first callback?

A Node.js convention where the first argument of the callback function is reserved for an error object (if the operation failed), and subsequent arguments hold the successful result data: `(err, data) => { if (err) { ... } }`.

10. What is a Buffer in Node.js?

A global class used to handle raw binary data directly in memory outside of V8's garbage-collected heap. Crucial for network streams and file I/O operations.

11. What causes the EADDRINUSE error and how do you resolve it?

It occurs when a server attempts to bind to a TCP port (e.g., 5050) that is already being used by another running process. Fix by killing the old process (`npx kill-port 5050` or `taskkill /F /PID ...`) or selecting a different port via `PORT=5051 node server.js`.

12. What does res.setHeader('Content-Type', 'application/json') do?

It tells the receiving client browser or API consumer that the incoming payload is formatted as a JSON string so that it can be parsed into a JavaScript object.

13. What is slugify and why was it used in Lab 08?

`slugify` is an npm utility that converts human-readable strings with spaces and special characters into clean, URL-safe lowercase slugs (e.g. `"Simple HTTP Server"` → `"simple-http-server"`).

14. What is the difference between path.join() and path.resolve()?

`path.join()` simply concatenates path segments using the OS separator (`\` on Windows, `/` on POSIX). `path.resolve()` resolves segments into an **absolute path** starting from the current working directory.

15. What are the key HTTP status code categories?

1xx: Informational | **2xx:** Success (200 OK, 201 Created) | **3xx:** Redirection (301 Moved, 304 Not Modified) | **4xx:** Client Error (400 Bad Request, 404 Not Found, 405 Method Not Allowed) | **5xx:** Server Error (500 Internal Server Error).

16. Why is blocking the event loop dangerous in Node.js?

Because Node.js executes JavaScript on a single thread. If an intensive synchronous CPU loop or synchronous file read blocks the thread, *every other connected client in the entire world is frozen* and cannot receive responses.

17. How do you prevent Node.js crashes from unhandled Promise rejections?

By attaching `.catch()` to every promise chain, using `try...catch` inside `async` functions, and setting up a global safety net: `process.on('unhandledRejection', (reason, promise) => { ... })`.

18. What is REPL in Node.js?

Read-Eval-Print Loop. An interactive terminal programming environment launched by typing `node` in the terminal to test JavaScript statements quickly.

19. How do you pass environment variables into a Node.js process?

Via terminal commands: `PORT=4000 STUDENT_NAME="Bhaskar Mall" node server.js` or cross-env. In code, they are accessed via the global object `process.env.VARIABLE_NAME`.

20. What is util.promisify?

A built-in Node.js utility that converts standard error-first callback functions (like `fs.readFile` or `child_process.execFile`) into functions that return ES6 Promises, enabling the use of `await`.

4. Live Exam Practical Execution & Commands Cheatsheet

Lab	Folder	Terminal Command to Run	Test URL / Verification
Lab 01	`Lab-01/`	`node server.js`	`http://localhost:3000/`
Lab 02	`Lab-02/`	`node server.js`	`http://localhost:3000/students`
Lab 03	`Lab-03/`	`node student-server.js` & `node food.js`	`http://localhost:3000/students?id=1`
Lab 04	`Lab-04/`	`node advanced-server.js`	`http://localhost:3000/students?course=BCA&minMarks=70`
Lab 05	`Lab-05/`	`node async-await-version.js`	Console shows step-by-step food delivery timestamps

Lab 06	`Lab-06/`	`node fs-demo.js`
		`node add-note.js "Prepare Viva"`
		`node read-notes.js`
			Inspects `notes.txt` file created on disk

Lab 07	`Lab-07/`	`node emitter.js`	Console verifies multi-listener and single-fire events
Lab 08	`Lab-08/`	`npm start` (or `node server.js`)	`http://localhost:3000/` & `http://localhost:3000/health`
Portfolio	`NodeJS-lab/`	`npm start` (or `node portfolio-server.js`)	`http://localhost:5050/` (Auto-Discovered Dashboard)

Faculty Demo Pro-Tip: If the teacher asks to test Lab-08:

1. Open terminal: `cd Lab-08`
2. Start server: `node server.js`
3. Open browser: `http://localhost:3000/` (shows all 8 integrated labs with local badge)
4. Show `http://localhost:3000/health` → returns `{"status":"ok", "environment":"local"}`
5. Show `http://localhost:3000/labs/02/app/students?course=BCA` → proves delegation to Lab 02 server handler!
6. Show `http://localhost:3000/labs/05/run` → proves `child_process.execFile` executing script labs!
7. Show `http://localhost:3000/api/dashboard` → proves `Promise.all` reading logs and screenshots concurrently!